

LEYAN LI

leyanpower@gmail.com | +44 7887 930137 | GitHub: github.com/lionlepower | leyan-li.com

Edinburgh, UK · open to relocation (UK / EU) · Right to work in the UK: Student visa (full-time work permitted post-course); applying for the 2-year Graduate Route — no sponsorship required.

PROFILE

MSc HPC & Data Science graduate targeting ML / AI infrastructure and performance engineering roles. Ran a 408-run hybrid MPI+OpenMP study on ARCHER2 (32 nodes / 4,096 cores), attributed layout rankings with Linaro MAP and PETSc -log_view, and built an evaluable LLM tool-calling agent on the data.

EDUCATION

University of Edinburgh — MSc High Performance Computing with Data Science | Sep 2025 – Dec 2026 (expected). Taught courses and dissertation completed Aug 2026 — **available immediately**; degree conferred Dec 2026

University of Liverpool / XJTLU — BSc Computer Science (2+2), **First-Class Honours** | Sep 2021 – Jun 2025

Relevant coursework: Message-Passing Programming (MPI), Threaded Programming (OpenMP), HPC Architectures, Performance Programming, GPU / Accelerator Programming, HPC Data Analytics, Machine Learning

TECHNICAL SKILLS

Languages: C / C++, Python, Bash, SQL

Parallel & performance: MPI, OpenMP, hybrid MPI+OpenMP, PETSc, Slurm, numactl / hwloc, Linaro MAP; strong / weak scaling, saturation; collective cost (halo, allreduce); roofline / memory bandwidth; NUMA, thread affinity

Deep learning & LLM: PyTorch, NumPy, pandas; LLM function calling / tool-schema design, offline evaluation and ablations. GPU and inference serving at coursework / self-study level: CUDA model, NCCL, data / tensor parallelism, batching vs. latency, KV cache, quantisation

Engineering: Linux, Git, pytest, GitHub Actions CI, FastAPI + Pydantic, SQLite / MySQL, working knowledge of Docker

PROJECTS

Parallel Layout Selection and Performance Attribution at 4,096 Cores on ARCHER2

| C · PETSc · MPI · OpenMP · Slurm · Linaro MAP · Python / pandas · CI lionlepower/HPC_PETSC_Benchmark

On ARCHER2, the UK national supercomputer (dual-socket AMD EPYC 7742, 128 cores/node, Slingshot): with total core count fixed, how should work split between MPI ranks and OpenMP threads per rank? Workload: 2D/3D stencil with CG+GAMG, 5M–165M unknowns, four full-node layouts from 128×1 (flat MPI) to 16×8.

- **Layout selection:** In 2D weak scaling, 64×2 beat flat MPI by 13.1% (794.8M equations/s at 165M unknowns on 32 nodes); fixed-20M strong scaling reached 29.83× speedup at 93.2% cumulative efficiency. In 3D the optimum moved to 32×4 (up to +33.2%) and 16×8 (3.81× faster than flat MPI): the best degree of parallelism shifts with dimension, size and per-core work — there is no single “optimal thread count”.
- **Root-cause attribution:** Linaro MAP explained the ranking as the crossing of two curves — as threads per rank rose, the MPI share of the solve window fell 46.7% → 6.2% while OpenMP wait rose 48.3% → 76.5%: communication exposure traded against thread idling. Ruled out threaded BLAS (LibSci ≤ 2.1% of solve-window core-time); as instrumentation perturbs runtime, the profile only ranked layouts and was never mixed with uninstrumented timings.
- **Additive metric of my own:** Defined a “synchronisation exposure share” on PETSc -log_view: event nesting makes naive leaf-event sums reach 116–372% of KSPSolve, but halo exchange and global reductions sit on disjoint call paths, so their sum is the one legitimate metric. Zero instrumentation overhead, covers all 264 weak-scaling runs; at 32 nodes it falls from 48.2% for flat MPI to 26.4% for 16×8.
- **Trustworthiness gate:** An audit found 302 of 303 logs whose actual MPI process count disagreed with the filename, so I discarded that entire phase of performance curves; formal conclusions rest on 408 audited runs alone. Made “parse process and thread counts from log content, not filenames” a hard check at the analysis entry point.

Agent-Based Performance Analysis System over the Benchmark Data

| Python · SQLite · FastAPI · Pydantic · OpenAI SDK (function calling) · pytest lionlepower/HPC_Agent

Structured the benchmark data above into SQLite and wrapped each analysis task as a deterministic tool: the LLM only decides which tool to call and with what arguments, while every number comes from deterministic code — turning agent correctness into something measurable.

- **Measurable routing:** Built a rule-based router as a verifiable baseline, output-isomorphic with the LLM function-calling router (intent + arguments), so both score on one self-built 30-question, five-category set — 83.3% for rules, 100% for the LLM. Ablation: the same model scores 63.3% without the system prompt, so accuracy is driven by tool documentation and policy prompt, not the model.
- **Defensive parsing and engineering:** Collapsed three failure modes — no tool call, hallucinated tool name, malformed JSON arguments — into an explicit unknown rather than guessing, so the system never returns a silently wrong number. A trace_id-threaded JSON Lines event stream replays any execution; FastAPI POST /ask maps an invalid router to 422 and a missing key to 503. 229 automated tests run fully offline.

Value Factorisation in Multi-Agent Actor-Critic Methods (BSc dissertation)

| Python · PyTorch · PyMARL · MAPPO / VDN / QMIX · PettingZoo lionlepower/MARL_VALUE_FACTORISATION

- **Counter-intuitive result and root cause:** Attached VDN additive factorisation and QMIX monotonic mixing to a MAPPO critic as a single-variable comparison (shared runner, buffer, actor and PPO hyperparameters; only the critic differs). The more expressive critic did not win — on sparse-reward Predator-Prey (300k steps) only the additive critic reached positive return (≈150); QMIX's critic loss opened at 5.4×10^4 and diverged again after 200k steps. Cause: the inherited value-based target $\max_a Q_i$ against an on-policy actor makes td_error systematically optimistic, not a valid advantage. The non-monotonic mixer's gradient explosion is reported as a negative result.

Publication: Li, Leyan. Convolutional Neural Networks Based Medical Image Analysis. EMITI 2024.